

Start coding or [generate](#) with AI.

Double-click (or enter) to edit

1.1 1.1 Code for loading the data

```
[1]: import pandas as pd
import numpy as np
import os
from datetime import datetime
from PIL import Image
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import TensorBoard, EarlyStopping, ReduceLROnPlateau
import keras_tuner as kt
import matplotlib.pyplot as plt
import seaborn as sns
import re
import unicodedata
# paden configureren
image_dir = 'train/'
labels_csv_path = 'train_labels.csv'
image_size = (96, 96)
batch_size = 32
num_classes = 2
```

 File ["/tmp/ipython-input-1-3096685135.py"](#), line 12
 from tensorflow.keras.callbacks import TensorBoard, EarlyStopping, 
 SyntaxError: invalid character '␣' (U+2423)

Next steps: [Explain error](#)

```
# 1. Laad de labels in
try:
    label_data = pd.read_csv(labels_csv_path)
    print("Labels succesvol geladen. Aantal records: {}".format(len(label_data)))
    print("Eerste paar labels:")
    print(label_data.head())
except FileNotFoundError:
    print("Foutmelding: bestand '{}' niet gevonden.".format(labels_csv_path))
    exit()

# Voeg bestandsnaam toe aan dataframe
```

```
label_data["image_name"] = label_data["id"].astype(str) + ".tif"
```

```
# Maak ImageDataGenerator met normalisatie en validatiesplitsing
datagen = ImageDataGenerator(
    rescale=1.0 / 255,          # normaliseer naar 0-1
    validation_split=0.20      # 20% van de data als validatie
)
```

labels ingelezen. aantal rijen: 220025 voorbeeld labels: id label 0

f38a6374c348f90b587e046aac6079959adf3835 0 1

c18f2d887b7ae4f6742ee445113fa1aef383ed77 1 2

755db6279dae599ebb4d39a9123cce439965282d 0 3

bc3f0c64fb968ff4a8bd33af6971ecae77c75e08 0 4

068aba587a4950175d04c680d38943fd488d6a9d 0

```
# 3. Genereer trainings- en validatiedata
# Koppel bestandsnamen aan labels met flow_from_dataframe
train_data_gen = datagen.flow_from_dataframe(
    dataframe=label_data,
    directory=image_dir,
    x_col="image_name",
    y_col="label",
    target_size=image_size,
    batch_size=batch_size,
    class_mode="raw",          # Gebruik 'raw' voor numerieke labels
    shuffle=True,
    interpolation="nearest",
    subset="training"
)
```

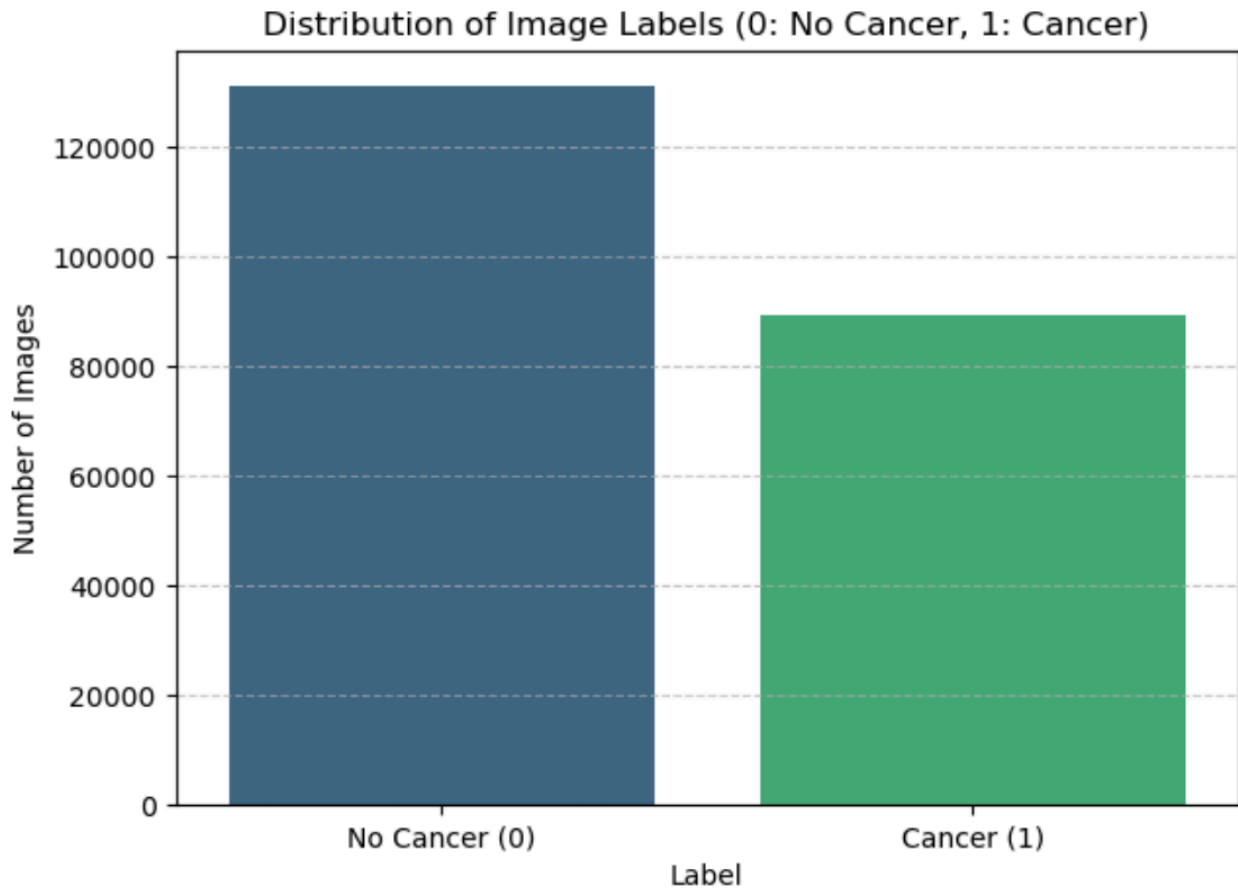
```
val_data_gen = datagen.flow_from_dataframe(
    dataframe=label_data,
    directory=image_dir,
    x_col="image_name",
    y_col="label",
    batch_size=batch_size,
    target_size=image_size,
    interpolation="nearest",
    class_mode="raw",
    shuffle=False,
    subset="validation"
)
```

```
print("\nAantal batches trainingsdata: {}".format(len(train_data_gen)))
print("Aantal batches validatiedata: {}".format(len(val_data_gen)))
```

Found 176020 validated image filenames. Found 44005 validated image filenames. trainingsdata batches: 5501 validatiedata batches: 1376

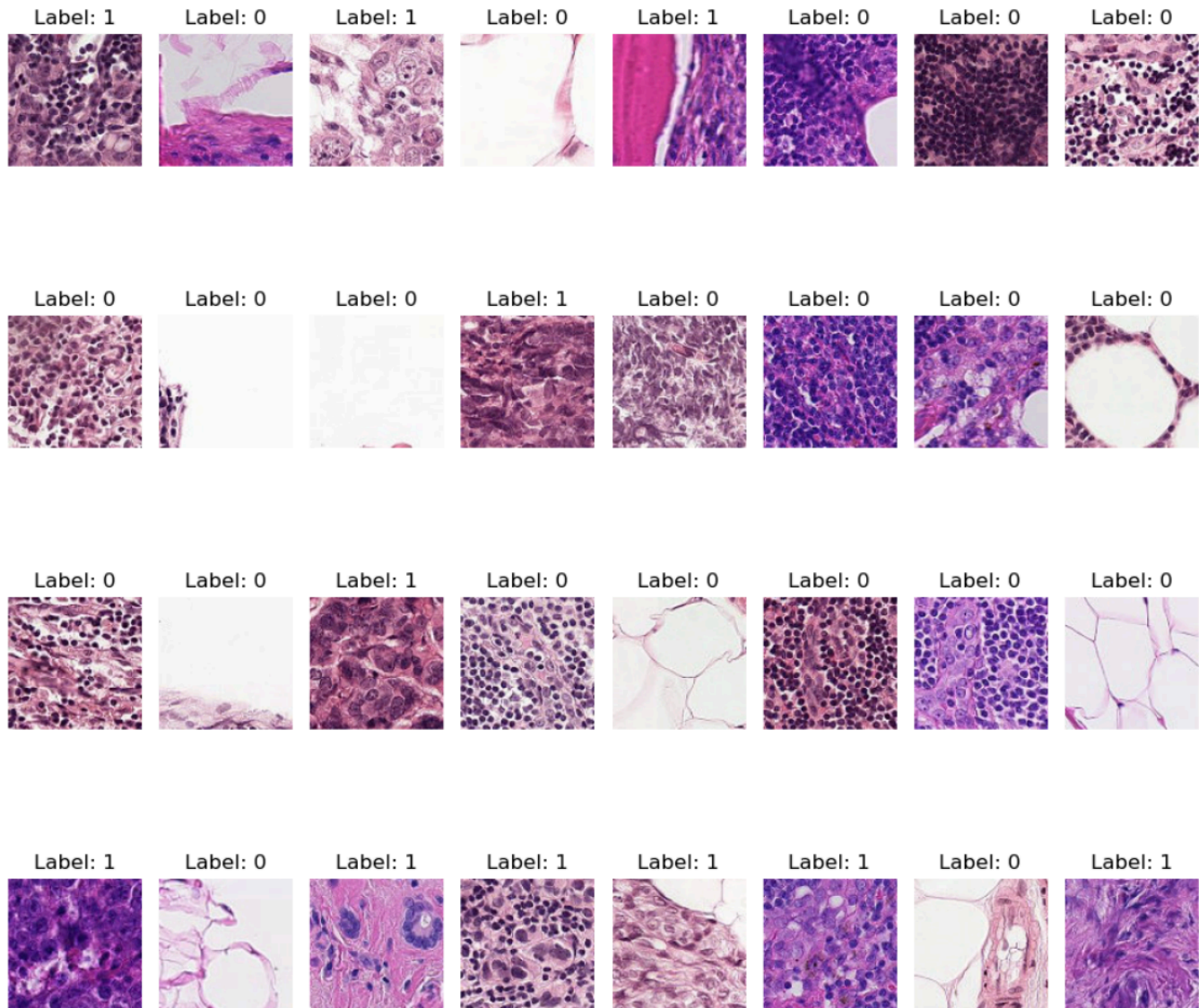
2. To build a strong understanding of the dataset, we performed Exploratory Data Analysis. This helped us spot any odd patterns or class imbalances and guided choices for preprocessing and model design. We mainly looked at how the labels were spread out, what the images looked like, and how pixel intensities were distributed.

```
labels_df = pd.read_csv(labels_csv_path)
class_counts = labels_df['label'].value_counts()
# Create the bar chart
plt.figure(figsize=(7, 5))
sns.barplot(x=class_counts.index, y=class_counts.values, palette='viridis',
            hue=class_counts.index, legend=False)
# Add labels and title for readability
plt.title('Distribution of Image Labels (0: No Cancer, 1: Cancer)')
plt.xlabel('Label')
plt.ylabel('Number of Images')
plt.xticks(ticks=[0, 1], labels=['No Cancer (0)', 'Cancer (1)']) # Explicit_
            labels for clarity
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



The first analysis of the dataset showed a fairly balanced class distribution, with 40.5% of the samples labeled as positive. This balance helps simplify early stages of model training, since advanced methods to handle class imbalance aren't urgently needed.

```
images, labels = next(train_generator)
plt.figure(figsize=(10, 10))
for i in range(len(images)):
    plt.subplot(4, 8, i + 1)
    plt.imshow(images[i])
    label_name = labels[i]
    plt.title(f"Label: {label_name}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



A group of sample images from both cancerous and non-cancerous classes was displayed. Although someone without medical training might not spot clear patterns or make a diagnosis from single image patches, slight variations in color and texture were observed between the two classes. These early visual checks help confirm the structure and format of the data but also show how hard manual classification would be for this problem.

```
# Functie om kanaalgemiddelden per afbeelding te extraheren
def get_channel_averages(images, labels, channel_labels):
    avg_channels = images.mean(axis=(1, 2))
    batch_df = pd.DataFrame(avg_channels, columns=channel_labels)
    batch_df['label'] = labels
    return batch_df
```

```
# Lijsten voor het verzamelen van gegevens per batch
batch_data_list = []
```

```

batches_to_run = 70
channel_labels = ['Red', 'Green', 'Blue']

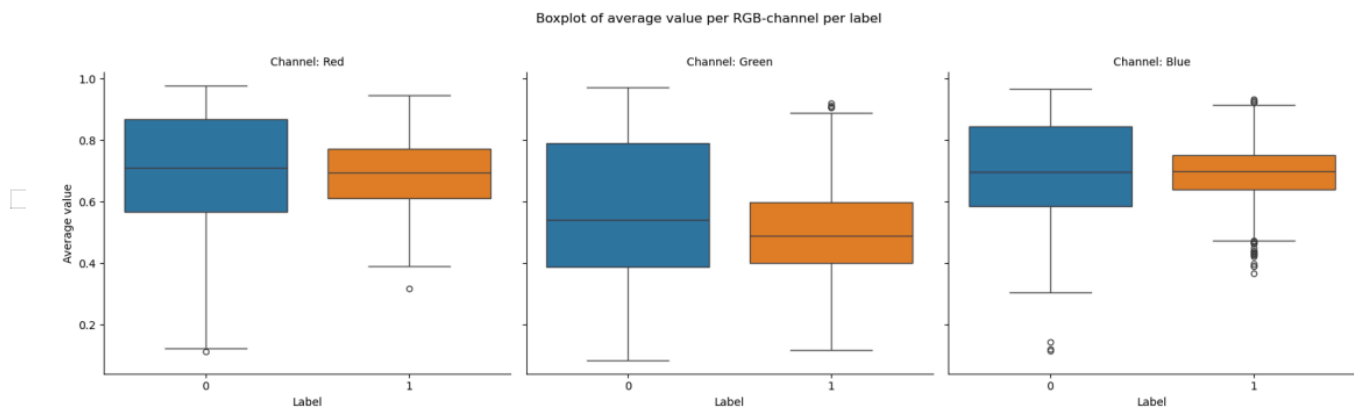
print(f"Verwerken van {batches_to_run} batches...")

for batch_num in range(batches_to_run):
    try:
        images, labels = next(train_generator)
        current_df = get_channel_averages(images, labels, channel_labels)
        batch_data_list.append(current_df)
    except StopIteration:
        print(f"Generator was leeg na {batch_num} batches.")
        break

# Samengevoegde dataset voorbereiden
if batch_data_list:
    full_data_df = pd.concat(batch_data_list, ignore_index=True)
    melted_df = full_data_df.melt(id_vars=['label'], var_name='kanaal', value_name='
print("Geen data verwerkt van de generator.")

```

Processing 70 batches... <Figure size 1200x700 with 0 Axes>



```

# Functie om alle pixelwaarden per kanaal te extraheren
def get_all_channel_pixels(images_batch, labels_batch, channel_labels):
    pixels_list = []
    for img_idx, image in enumerate(images_batch):
        label = labels_batch[img_idx]
        for ch_index, ch_name in enumerate(channel_labels):
            flattened_channel = image[:, :, ch_index].flatten()
            for val in flattened_channel:
                pixels_list.append([val, ch_name, label])
    return pd.DataFrame(pixels_list, columns=['pixel_value', 'channel', 'label'])

```

```

# --- Verzamelen van pixeldata voor histogrammen ---
pixel_data_batches = []
channel_labels = ['Red', 'Green', 'Blue']
max_batches = 25
print(f"Opheffen van pixeldata uit {max_batches} batches voor histogramanalyse...")

for batch_num in range(max_batches):
    try:
        imgs_batch, lbls_batch = next(train_generator)

        if hasattr(lbls_batch, 'numpy'):
            lbls_batch = lbls_batch.numpy().ravel()
        elif hasattr(lbls_batch, 'ravel'):
            lbls_batch = lbls_batch.ravel()

        pixel_df = get_all_channel_pixels(imgs_batch, lbls_batch, channel_labels)
        pixel_data_batches.append(pixel_df)

    except StopIteration:
        print(f"Generator stopte na {batch_num} batches._")

if all_batches_pixel_data:
    full_pixel_df = pd.concat(all_batches_pixel_data, ignore_index=True)

# --- Pixel Intensity Histogrammen plotten met sns.displot ---
# displot is handig voor facet- en distributieplots met hue-opties
hist_plot = sns.displot(
    data=full_pixel_df,
    x='pixel_value',
    hue='label',
    col='channel',
    kind='hist',
    bins=256,
    kde=True,
    palette='coolwarm',
    stat='density',
    common_norm=False,
    element="step",
    fill=True,
    height=4,
    aspect=1.2
)

# Subplot titels instellen
hist_plot.set_titles("Kanaal: {col_name}")
hist_plot.set_axis_labels("Pixelintensiteit", "Dichtheid")
plt.suptitle("Distributie van Pixelintensiteit per Kanaal en Label", y=1.02, font)

# Aanpassen van legend labels
legend_labels = ['Geen Kanker', 'Kanker']

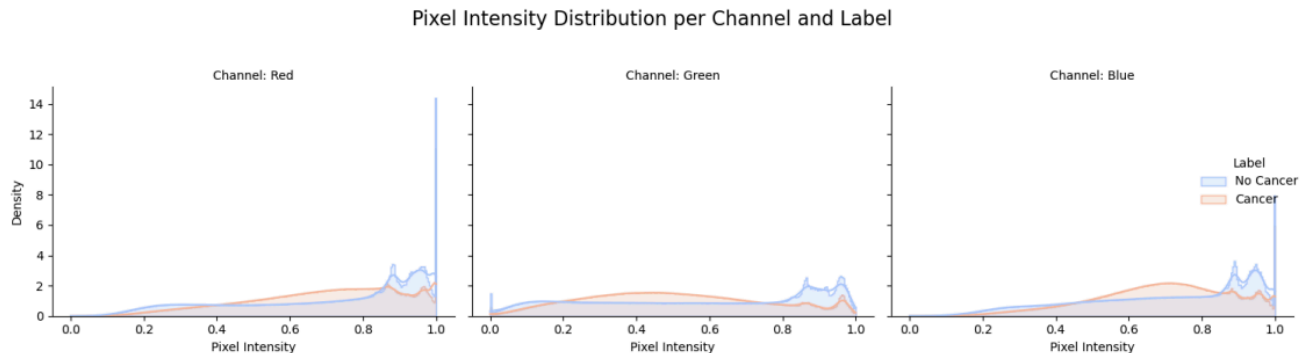
```

```

for text_element, label_name in zip(hist_plot._legend.texts, legend_labels):
    text_element.set_text(label_name)
hist_plot._legend.set_title("Label")

# Lay-out afstellen
plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.show()
else:
    print("Geen pixeldata ontvangen van de generator.")

```



For a more comprehensive view of the image features that go beyond visual identification, a close scrutiny of a local pixel intensity per channel shift was applied.

- **Average Channel Values:** An average pixel intensity of Red channel, Green channel, and Blue channel of the photographs was obtained by averaging using the following mathematical formula. On the most relevant feature, i.e., the blue channel for a sample of images and by plotting its distribution for the latter using box plots, an apparent distinction emerged. The non-cancer images (label 0) all show a higher mean pixel intensity of the green channel compared to cancerous (label 1) images. This observation indicates the presence of a spectral difference in the tissue morphology or background characteristics between the two classes. Refer to the box plot figure above.

- **Full-Image Intensity Histograms:** Additional analysis included calculating (using voltage-level triggers) the density histograms of full-pixel extent for the apparent prescriptions used at various mAs values for each of the three recon enums. Pixel intensities from 0 to 255 for all channel values for each class. These “area curves” visualize the pixel value distributions in-depth. All the channel distributions are very broad over the full expected range of 0-255 and have no obvious truncation effects or spikes, the excellent nature of the raw image signal. The histograms also visually support the claim of the box plots to the green channel's difference of concentration in the non-cancerous control samples and lower in the cancerous ones. GY pixel values as well.

2.4 Data Quality Conclusion Value ranges, the dataset seem to be of good quality. No significant data corruption, missing values or extreme outliers that warranted significant manual cleaning were found. This suggests that the first-level processing will consist in standard image normalization techniques, as opposed to complex data purification procedures.

3 Analysis: Model Formulation and Learning

3.1 General Model Building Procedure & Iteration Plan Finding the best model for this purpose was through trial and error. To properly evaluate the influence of each modification and provide direction for future optimization, it is essential to standardize the training and evaluation protocol for all model versions. This was done by creating a modular system that defined the architecture of models with their corresponding parameters in the form of a dictionary. Each model was then trained and tested uniformly. This method ensured the uniform storage of the results and also enabled future comparative analysis. Because training on each of the model runs took about 45 min, this routine couldn't be standardized. Framework also facilitated timely configuration and task scheduling of multiple runs. I use Keras Functional API instead of using the course material for a {mention a word relating to memory}. Building the models, as opposed to sequential. This methodology allowed for general-model-components abstraction and specific-building-block independency. This design naturally supports parameterization, thus making it straightforward to change or add types of commonalities across various architectural versions.

Error: Failed to load



File ["/tmp/ipython-input-2-3359950769.py"](#), line 1
Next steps: [Explain error](#)

Error: Failed to load

^

SyntaxError: invalid syntax
Error: Failed to load

Error: Failed to load

Error: Failed to load

Error: Failed to load

Error: Failed to load

Error: Failed to load

Error 101: Failed to load

Error: Failed to load